



TRƯỜNG ĐẠI HỌC
QUỐC TẾ
MIỀN ĐÔNG

EASTERN
INTERNATIONAL
UNIVERSITY

BECAMEX BUSINESS SCHOOL

ACTIVITY 1

Banking Database SQL Practice Exercise

Course: MIS 443 - BUSINESS DATA MANAGEMENT

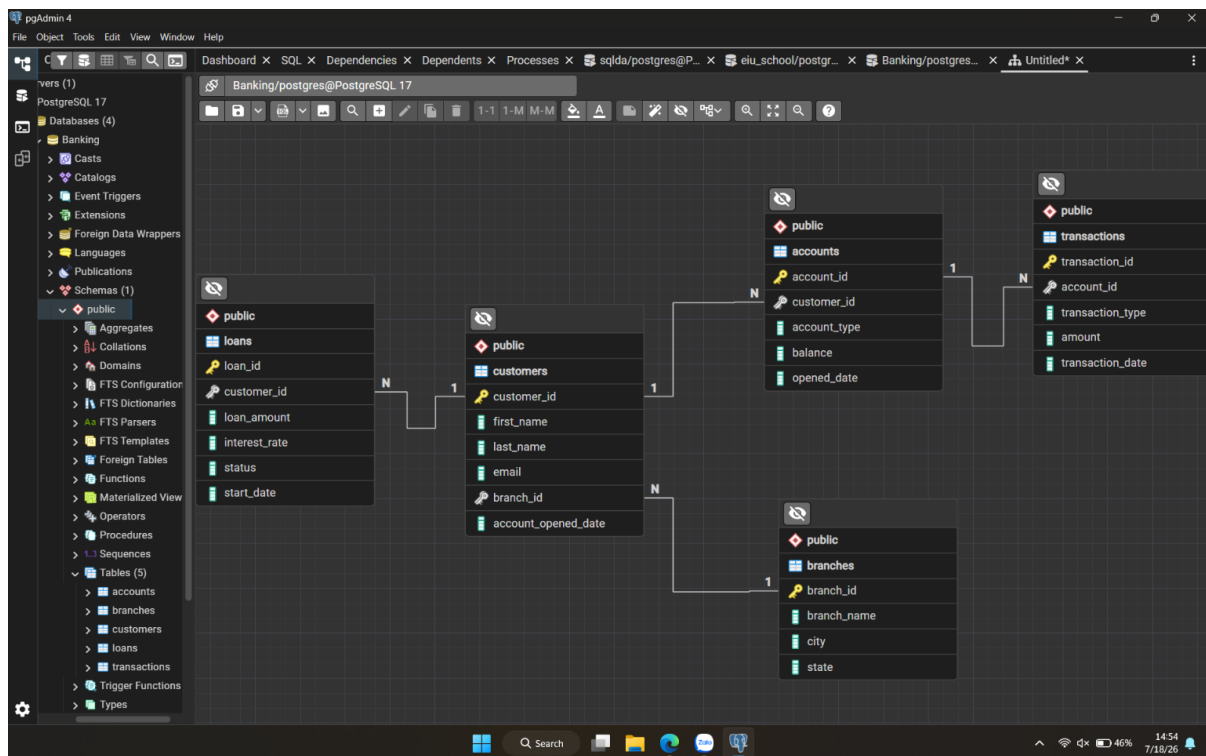
Lecturer: Mr. Dang Thai Doan

Group Name: D2NB

No.	Student's Name	Student's IRN
1	Vũ Đông Dương	2032300044
2	Thân Quế Ngọc	2232300060
3	Văn Vũ Quỳnh Như	2232300079
4	Đỗ Hoàng Bảo	2232300071

Quarter 4, Academic Year 2025-2026

Banking Database



This dataset simulates the database of a banking management system, made up of five related tables.

The "branches" table stores information about bank branches. The "customers" table holds customer information, with each customer linked to a branch through `branch_id`. The "accounts" table manages customer accounts through `customer_id`, while the "loans" table records loan information, also linked by `customer_id`.

Finally, the "transactions" table stores details of deposit and withdrawal transactions, tied to each account through `account_id`. This foreign key structure lets you trace from a customer at a given branch through to their full set of accounts, loans, and transaction history.

SQL Query Practice

Q1. Return the complete customer roster from the customers table.

The screenshot shows the pgAdmin 4 interface. The Query History pane displays a SQL query: `select * from customers;`. The Data Output pane shows the results of the query, which are 7 rows of customer data. The results are highlighted with a red box.

customer_id (PK) integer	first_name character varying (50)	last_name character varying (50)	email character varying (100)	branch_id integer	account_opened_date date	
1	1001	Alice	Smith	alice.smith@email.com	1	2020-03-15
2	1002	Bob	Johnson	bob.j@email.com	1	2019-07-22
3	1003	Carol	Williams	carol.w@email.com	2	2021-01-10
4	1004	David	Brown	david.b@email.com	2	2020-11-05
5	1005	Eva	Davis	eva.d@email.com	3	2022-06-18
6	1006	Frank	Miller	frank.m@email.com	3	2018-09-30
7	1007	George	Wilson	george.w@email.com	5	2025-03-01

Q2. Return all branch names and their cities.

The screenshot shows the pgAdmin 4 interface. The Query History pane displays a SQL query: `SELECT branch_name, city FROM branches;`. The Data Output pane shows the results of the query, which are 5 rows of branch data. The results are highlighted with a red box.

branch_name character varying (100)	city character varying (50)	
1	Main Street	New York
2	Downtown	Los Angeles
3	Central	Chicago
4	Riverside	Houston
5	Plaza	Phoenix

Q3. Return all accounts with account type Savings.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'Banking/postgres@PostgreSQL 17', including tables like 'accounts', 'branches', 'customers', 'loans', 'transaction', and 'types'. The main pane shows a SQL query editor with five queries. Query 3 is highlighted with a yellow box:

```
/* 3. Return all accounts with account type Savings.*/
SELECT accounts.account_id, accounts.customer_id, accounts.balance
FROM accounts
WHERE account_type = 'Savings';
```

The 'Data Output' tab shows the results of Query 3, which are highlighted with a blue box:

account_id [PK] integer	customer_id integer	balance real
1	5002	1001
2	5004	1003
3	5008	1006

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.357'.

Q4. Return accounts with a balance greater than \$10,000.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'Banking/postgres@PostgreSQL 18', including tables like 'accounts', 'branches', 'customers', 'loans', 'transaction', and 'types'. The main pane shows a SQL query editor with four queries. Query 4 is highlighted with a red box:

```
-- Q4. Return accounts with a balance greater than $10,000.
select customer_id, account_type, balance from accounts
where balance > 10000
order by balance;
```

The 'Data Output' tab shows the results of Query 4, which are highlighted with a red box:

customer_id integer	account_type character varying (20)	balance real
1	1001 Savings	12000
2	1004 CD	15000
3	1006 Savings	25000

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.066'.

Q5. Return all transactions of type Deposit.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including a 'transactions' table. The main query editor contains the following SQL code:

```

76 FROM accounts
77 WHERE balance > 10000
78 ORDER BY balance;
79 /*Q5: The operations team is reconciling all inbound cash flows and needs a full list of deposit transactions.
80 Return all transactions of type Deposit.
81 Output: Return transaction_id, account_id, amount, and transaction_date.
82 Rules: Only include transactions where transaction type = 'Deposit', Order by transaction_date.*/
83 SELECT transaction_id, account_id, amount, transaction_date
84 FROM transactions
85 WHERE transaction_type = 'Deposit'
86 ORDER BY transaction_date;
87
88 /*Q6: The credit team is reviewing all currently active loan obligations for quarterly reporting.
89 Return all loans with an Active status.
90 Output: Return loan_id, customer_id, loan_amount, and interest_rate.
91 Rules: Only include loans where status = 'Active'.*/
92 SELECT loan_id, customer_id, loan_amount, interest_rate

```

The 'Data Output' tab shows the results of the Q5 query:

transaction_id	account_id	amount	transaction_date
1	1	5001	2025-01-05
2	3	5002	2025-01-10
3	5	5004	2025-01-15
4	6	5005	2025-01-18
5	7	5001	2025-01-20

Total rows: 5. Query complete 00:00:00.120.

Q6. Return all loans with an Active status.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including a 'loans' table. The main query editor contains the following SQL code:

```

15 -- Q4: Return accounts with a balance greater than $10,000.
16 SELECT customer_id, account_type, balance
17 FROM accounts
18 WHERE balance >= 10000;
19
20 -- Q5: Return all transactions of type Deposit.
21 SELECT transaction_id, account_id, amount, transaction_date
22 FROM transactions
23 WHERE transaction_type = 'Deposit';
24
25 -- Q6: Return all loans with an Active status.
26 SELECT loan_id, customer_id, loan_amount, interest_rate
27 FROM loans
28 WHERE status = 'Active';
29
30
31
32

```

The 'Data Output' tab shows the results of the Q6 query:

loan_id	customer_id	loan_amount	interest_rate
1	1	15000	5.5
2	2	25000	6
3	4	50000	5.25

Total rows: 3. Query complete 00:00:00.425. Successfully run. Total query run.

Q7. Count the total number of accounts.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'Banking/postgres@PostgreSQL 17', including tables like 'accounts', 'branches', 'customers', 'loans', 'transaction', and 'types'. The main query editor contains the following SQL code:

```
/* 6. Return all loans with an Active status.*/
SELECT loan_id, customer_id, loan_amount, interest_rate
FROM loans
WHERE status = 'Active'
ORDER BY loan_id;

/* 7. Count the total number of accounts.*/
SELECT COUNT(*) AS total_accounts
FROM accounts;

/* 8. Sum the total amount across all Deposit transactions.*/
SELECT SUM(amount) AS total_deposits
FROM transactions
WHERE transaction_type = 'Deposit';

/* 9. The product team needs a list of all account types offered.
Find all unique account types available. Show account_type ordered alphabetically.*/
SELECT DISTINCT account_type
FROM accounts
ORDER BY account_type;

/* 10. The audit team needs to review mid-month activity. Find all transactions where
```

The 'Data Output' tab shows the result of the query for Q7:

total_accounts
8

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.113'.

Q8. Sum the total amount across all Deposit transactions.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'banking/postgres@PostgreSQL 18', including tables like 'accounts', 'branches', 'customers', 'loans', 'transactions', 'trigger functions', 'types', and 'views'. The main query editor contains the following SQL code:

```
-- Q5. Return all transactions of type Deposit.
select transaction_id, account_id, amount, transaction_date from transactions
where transaction_type = 'Deposit'
order by transaction_date;

-- Q6. Return all loans with an Active status.
select loan_id, customer_id, loan_amount, interest_rate from loans
where status = 'Active';

-- Q7. Count the total number of accounts.
select count(*) as total_accounts from ACCOUNTS;

-- Q8. Sum the total amount across all Deposit transactions.
select sum(amount) as total_deposits from transactions
where transaction_type = 'Deposit';
```

The 'Data Output' tab shows the result of the query for Q8:

total_deposits
2800

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.054'.

Q9. The product team needs a list of all account types offered. Find all unique account types available. Show account_type ordered alphabetically.

The screenshot shows the pgAdmin 4 interface. The SQL editor contains the following query:

```
SELECT DISTINCT account_type
FROM accounts
ORDER BY account_type;
```

The query is highlighted with a red box. The results pane shows the following data:

account_type
CD
Checking
Savings

The status bar indicates: "Successfully run. Total query runtime: 148 msec. 3 rows affected."

Q10. The audit team needs to review mid-month activity. Find all transactions where transaction_date is between January 10 and January 20, 2025 (inclusive). Show transaction_id, account_id, amount, and transaction_date. Order by transaction_date.

The screenshot shows the pgAdmin 4 interface. The SQL editor contains the following query:

```
SELECT transaction_id, account_id, amount, transaction_date
FROM transactions
WHERE transaction_date >= '2025-01-10' AND transaction_date <= '2025-01-20'
ORDER BY transaction_date ASC;
```

The query is highlighted with a red box. The results pane shows the following data:

transaction_id	account_id	amount	transaction_date
1	3	5002	2025-01-10
2	4	5003	2025-01-12
3	5	5004	2025-01-15
4	6	5005	2025-01-18
5	7	5001	2025-01-20

The status bar indicates: "Successfully run. Total query runtime: 135 msec. 5 rows affected."

Q11. The new-accounts team wants to follow up with registered customers who have not yet opened any account. Find all customers who have no accounts. Show first name and last name.

The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query for Q11 is highlighted in a yellow box:

```

/* 11. The new-accounts team wants to follow up with registered customers who have not yet opened any account. Find all customers who have no accounts. Show first name and last name.*/
SELECT c.first_name, c.last_name
FROM customers c
LEFT JOIN accounts a ON c.customer_id = a.customer_id
WHERE a.account_id IS NULL
ORDER BY c.last_name;

```

The Data Output tab shows the results of the query:

first_name	last_name
George	Wilson

The status bar indicates "Total rows: 1" and "Query complete 00:00:00.102".

Q12. The loans department needs to identify customers who have not taken any loan — potential targets for a new loan campaign. Find all customers who have no loans. Show first name and last name ordered by last name.

The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query for Q12 is highlighted in a red box:

```

-- Q12. The loans department needs to identify customers who have not taken any loan — potential targets
select first_name, last_name from customers
left join loans on customers.customer_id = loans.customer_id
where loans.customer_id is null
order by last_name;

```

The Data Output tab shows the results of the query:

first_name	last_name
Eva	Davis
Carol	Williams
George	Wilson

The status bar indicates "Total rows: 3" and "Query complete 00:00:00.082".

Q13. The relationship management team wants to know how diversified each customer's portfolio is. Show every customer's full name and the number of accounts they hold (including zero). Order by account_count descending, then last name.

The screenshot shows the pgAdmin 4 interface with a SQL query for Q13. The query is as follows:

```

128 FROM customers c
129 LEFT JOIN loans l ON c.customer_id = l.customer_id
130 WHERE l.loan_id IS NULL
131 ORDER BY c.last_name;
/*Q13: The relationship management team wants to know how diversified each customer's portfolio is. Show every customer's full name and the number of accounts they hold (including zero). Order by account_count descending, then last name.*/
134 SELECT c.first_name, c.last_name, COUNT(a.account_id) AS account_count
135 FROM customers c
136 LEFT JOIN accounts a ON c.customer_id = a.customer_id
137 GROUP BY c.customer_id, c.first_name, c.last_name
138 ORDER BY account_count DESC, c.last_name;
/*Q14: The finance team is preparing a balance summary broken down by account product type. Calculate the total balance for each account type.*/

```

The Data Output tab shows the following results:

first_name	last_name	account_count
David	Brown	2
Alice	Smith	2
Eva	Davis	1
Bob	Johnson	1
Frank	Miller	1
Carol	Williams	1
George	Wilson	0

Successfully run. Total query runtime: 215 msec. 7 rows affected.

Q14. Calculate the total balance for each account type.

The screenshot shows the pgAdmin 4 interface with a SQL query for Q14. The query is as follows:

```

77 SELECT first_name, last_name, COUNT(account_id) AS account_count
78 FROM customers
79 LEFT JOIN accounts ON customers.customer_id = accounts.customer_id
80 GROUP BY first_name, last_name, customers.customer_id
81 ORDER BY account_count DESC, last_name;
--Q14: Calculate the total balance for each account type.
84 SELECT SUM(balance) AS total_balance, account_type
85 FROM accounts
86 GROUP BY account_type
87 ORDER BY account_type;
--Q15: Return each customer's name alongside their branch name.
90 SELECT first_name, last_name, branch_name
91 FROM customers
92 LEFT JOIN branches ON customers.branch_id = branches.branch_id
93 ORDER BY last_name;
--Q16: Return each transaction with the transaction date, amount, type, and the customer's last name.
95

```

The Data Output tab shows the following results:

total_balance	account_type
15000	CD
11601.5	Checking
45500	Savings

Successfully run. Total query runtime: 224 msec. 3 rows affected.

Q15. Return each customer's name alongside their branch name.

The screenshot shows pgAdmin 4 with a SQL query window titled 'Banking/postgres@PostgreSQL 17'. The query is as follows:

```

/* 15. Return each customer's name alongside their branch name.*/
SELECT c.first_name, c.last_name, b.branch_name
FROM customers c JOIN branches b
ON b.branch_id = c.branch_id
ORDER BY last_name;

```

The 'Data Output' tab shows the results of the query:

first_name	last_name	branch_name
David	Brown	Downtown
Eva	Davis	Central
Bob	Johnson	Main Street
Frank	Miller	Central
Alice	Smith	Main Street
Carol	Williams	Downtown
George	Wilson	Plaza

Total rows: 7. Query complete 00:00:00.111.

Q16. Return each transaction with the transaction date, amount, type, and the customer's last name.

The screenshot shows pgAdmin 4 with a SQL query window titled 'banking/postgres@PostgreSQL 18'. The query is as follows:

```

-- Q16. Return each transaction with the transaction date, amount, type, and the customer's last name.
select transaction_date, amount, transaction_type, last_name from transactions
join accounts on transactions.account_id = accounts.account_id
join customers on accounts.customer_id = customers.customer_id
order by transaction_date;

```

The 'Data Output' tab shows the results of the query:

transaction_date	amount	transaction_type	last_name
2025-01-05	500	Deposit	Smith
2025-01-08	200	Withdrawal	Smith
2025-01-10	1000	Deposit	Smith
2025-01-12	150	Withdrawal	Johnson
2025-01-15	250	Deposit	Williams
2025-01-18	750	Deposit	Brown
2025-01-20	300	Deposit	Smith
2025-01-22	100	Withdrawal	Davis

Total rows: 8. Query complete 00:00:00.054. Successfully run. Total query run...

Q17. Return the number of customers assigned to each branch.

The screenshot shows the pgAdmin 4 interface. The Query History pane displays a SQL query for Q17: "Return the number of customers assigned to each branch." The query is as follows:

```
SELECT b.branch_name, COUNT(c.customer_id) AS customer_count
FROM branches b
LEFT JOIN customers c ON b.branch_id = c.branch_id
GROUP BY b.branch_id, b.branch_name
ORDER BY b.branch_id;
```

The Data Output pane shows the results of the query:

branch_name	customer_count
1 Main Street	2
2 Downtown	2
3 Central	2
4 Riverside	0
5 Plaza	1

The status bar indicates: "Successfully run. Total query runtime: 149 msec. 5 rows affected."

Q18. Return the number of accounts held by each customer.

The screenshot shows the pgAdmin 4 interface. The Query History pane displays a SQL query for Q18: "Return the number of accounts held by each customer." The query is as follows:

```
SELECT customer_id, COUNT(account_id) AS account_count
FROM accounts
GROUP BY customer_id
ORDER BY customer_id;
```

The Data Output pane shows the results of the query:

customer_id	account_count
1 1001	2
2 1002	1
3 1003	1
4 1004	2
5 1005	1

The status bar indicates: "Query complete 00:00:00.122".

Q19. Return each loan with the borrower's first and last name, loan amount, and status.

The screenshot shows the pgAdmin 4 interface. The SQL editor contains the following query for Q19:

```

GROUP BY a.customer_id
ORDER BY a.customer_id;

/* 19. Return each loan with the borrower's first and last name, loan amount, and status.*/
SELECT c.first_name, c.last_name, l.loan_amount, l.status FROM loans l
JOIN customers c ON l.customer_id = c.customer_id
ORDER BY l.loan_amount DESC, c.last_name;

/* 20. Return the total account balance held by customers at each branch.*/
SELECT b.branch_name, SUM(a.balance) AS total_balance
FROM branches b
JOIN customers c ON c.branch_id = b.branch_id
JOIN accounts a ON c.customer_id = a.customer_id
GROUP BY b.branch_name
ORDER BY total_balance DESC;

/* 21. The branch operations team wants a portfolio view of every customer's holdings.
Show each customer's name, their branch name, account type, and balance.
Order by branch name, then customer last name.*/

```

The Data Output pane shows the results of the Q19 query:

	first_name character varying (50)	last_name character varying (50)	loan_amount real	status character varying (20)
1	Frank	Miller	50000	Active
2	Bob	Johnson	25000	Active
3	Alice	Smith	15000	Active
4	David	Brown	10000	Paid

Message: Successfully run. Total query runtime: 87 msec. 4 rows affected.

Q20. Return the total account balance held by customers at each branch.

The screenshot shows the pgAdmin 4 interface. The SQL editor contains the following query for Q20:

```

-- Q20. Return the total account balance held by customers at each branch.
select branch_name, sum(balance) as total_balance from branches
join customers on branches.branch_id = customers.branch_id
join accounts on customers.customer_id = accounts.customer_id
group by branch_name
order by total_balance desc;

-- Q21. The branch operations team wants a portfolio view of every customer's holdings. Show each custom
select first_name, last_name, branch_name, account_type, balance from customers

```

The Data Output pane shows the results of the Q20 query:

	branch_name character varying (100)	total_balance real
1	Downtown	27700.25
2	Central	26800
3	Main Street	17601.25

Message: Query complete 00:00:00.107

Q21. The branch operations team wants a portfolio view of every customer's holdings. Show each customer's name, their branch name, account type, and balance. Order by branch name, then customer last name.

The screenshot shows the pgAdmin 4 interface with a SQL query window. The query is as follows:

```

SELECT c.first_name, c.last_name, b.branch_name, a.account_type, a.balance
FROM customers c
JOIN accounts a ON c.customer_id = a.customer_id
JOIN branches b ON c.branch_id = b.branch_id
ORDER BY b.branch_name, c.last_name;

```

The results are displayed in a table with 8 rows:

	first_name	last_name	branch_name	account_type	balance
1	Eva	Davis	Central	Checking	1800
2	Frank	Miller	Central	Savings	25000
3	David	Brown	Downtown	Checking	4200.25
4	David	Brown	Downtown	CD	15000
5	Carol	Williams	Downtown	Savings	8500
6	Bob	Johnson	Main Street	Checking	2100.75
7	Alice	Smith	Main Street	Checking	3500.5

Message: Successfully run. Total query runtime: 144 msec. 8 rows affected.

Q22. For each account that has transactions, show total deposits and total withdrawals side by side using conditional aggregation. Order by account_id.

The screenshot shows the pgAdmin 4 interface with a SQL query window. The query is as follows:

```

SELECT account_id,
SUM(CASE WHEN transaction_type = 'Deposit' THEN amount ELSE 0 END) AS total_deposits,
SUM(CASE WHEN transaction_type = 'Withdrawal' THEN amount ELSE 0 END) AS total_withdrawals
FROM transactions
GROUP BY account_id
ORDER BY account_id;

```

The results are displayed in a table with 6 rows:

	account_id	total_deposits	total_withdrawals
1	5001	800	200
2	5002	1000	0
3	5003	0	150
4	5004	250	0
5	5005	750	0

Message: Successfully run. Total query runtime: 128 msec. 6 rows affected.

Q23. Group transactions by year-month (using strftime). Show month, transaction count, and total amount. Order by month ascending.

Using the `to\_char` function because it is the standard function for formatting dates in PostgreSQL. `strftime` is a function that is only supported by the SQLite database management system.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'PostgreSQL 17', including 'Banking' and 'public' schemas. The main pane shows a SQL query for Q23, which is highlighted with a yellow box. The query is as follows:

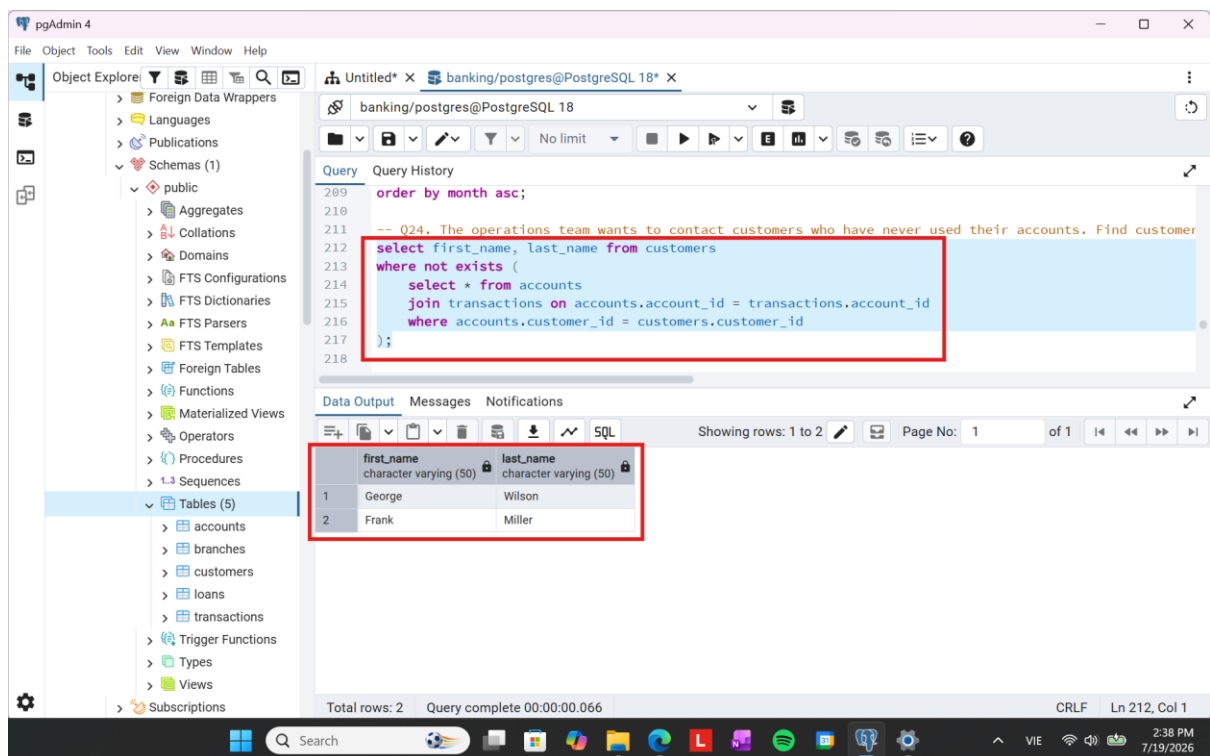
```
/*23. Group transactions by year-month (using strftime).  
Show month, transaction count, and total amount. Order by month ascending.*/  
SELECT TO_CHAR(transaction_date, 'YYYY-MM') AS month,  
       COUNT(transaction_id) AS transaction_count,  
       SUM(amount) AS total_amount  
FROM transactions  
GROUP BY month  
ORDER BY month ASC;
```

Below the query editor, the 'Data Output' tab is active, showing a table with the following data:

month	transaction_count	total_amount
2025-01	0	3250

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.131'. A green message box at the bottom right states 'Successfully run. Total query runtime: 131 msec. 1 rows affected.'

Q24. The operations team wants to contact customers who have never used their accounts. Find customers who have no transactions in any of their accounts. Show first name and last name.



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including a 'public' schema with various objects and a 'Tables (5)' section containing 'accounts', 'branches', 'customers', 'loans', and 'transactions'. The central pane shows a SQL query for Q24, which is highlighted with a red box:

```

order by month asc;
-- Q24. The operations team wants to contact customers who have never used their accounts. Find customer
select first_name, last_name from customers
where not exists (
  select * from accounts
  join transactions on accounts.account_id = transactions.account_id
  where accounts.customer_id = customers.customer_id
);

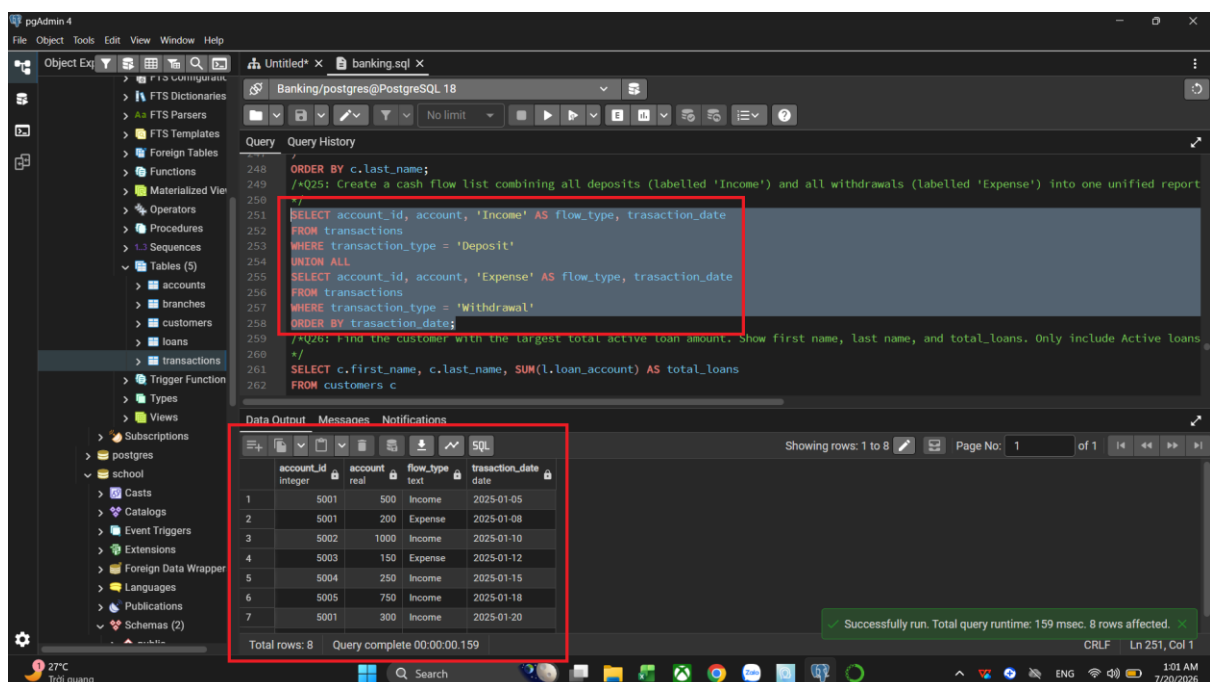
```

The bottom pane shows the query results in a table format, also highlighted with a red box:

	first_name	last_name
1	George	Wilson
2	Frank	Miller

The status bar at the bottom indicates 'Total rows: 2' and 'Query complete 00:00:00.066'.

Q25. Create a cash flow list combining all deposits (labelled 'Income') and all withdrawals (labelled 'Expense') into one unified report. Show account_id, amount, flow_type, and transaction_date. Order by transaction_date.



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including a 'public' schema with various objects and a 'Tables (5)' section containing 'accounts', 'branches', 'customers', 'loans', and 'transactions'. The central pane shows a SQL query for Q25, which is highlighted with a red box:

```

ORDER BY c.last_name;
/*Q25: Create a cash flow list combining all deposits (labelled 'Income') and all withdrawals (labelled 'Expense') into one unified report
SELECT account_id, account, 'Income' AS flow_type, transaction_date
FROM transactions
WHERE transaction_type = 'Deposit'
UNION ALL
SELECT account_id, account, 'Expense' AS flow_type, transaction_date
FROM transactions
WHERE transaction_type = 'Withdrawal'
ORDER BY transaction_date;
*/
/*Q26: Find the customer with the largest total active loan amount. Show first name, last name, and total_loans. Only include Active loans.
SELECT c.first_name, c.last_name, SUM(l.loan_amount) AS total_loans
FROM customers c

```

The bottom pane shows the query results in a table format, also highlighted with a red box:

	account_id	account	flow_type	transaction_date
1	5001	500	Income	2025-01-05
2	5001	200	Expense	2025-01-08
3	5002	1000	Income	2025-01-10
4	5003	150	Expense	2025-01-12
5	5004	250	Income	2025-01-15
6	5005	750	Income	2025-01-18
7	5001	300	Income	2025-01-20

The status bar at the bottom indicates 'Total rows: 8' and 'Query complete 00:00:00.159'.

Q26. Find the customer with the largest total active loan amount. Show first name, last name, and total_loans. Only include Active loans.

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure for 'BANK_PRACTICE'. The central pane shows a SQL query for Q26, which is highlighted with a red box. The query is as follows:

```

CASE
WHEN transaction_type = 'Deposit' THEN 'Income'
ELSE 'Expense'
END AS flow_type
FROM transactions
ORDER BY transaction_date;

-- Q26: Find the customer with the largest total active loan amount.
-- Show first_name, last_name, and total_loans. Only include Active loans.
SELECT c.first_name, c.last_name, SUM(l.loan_amount) AS total_loans
FROM customers c
INNER JOIN loans l
ON c.customer_id = l.customer_id
WHERE l.status = 'Active'
GROUP BY c.first_name, c.last_name
ORDER BY total_loans DESC
LIMIT 1;

```

The bottom pane shows the 'Data Output' tab with the result of the query, also highlighted with a red box. The result is a single row:

first_name	last_name	total_loans
Frank	Miller	50000

The status bar at the bottom indicates 'Successfully run. Total query runtime: 129 msec. 1 rows affected.'

Q27. The wealth management team wants to identify accounts performing above their peer group. Find accounts with a balance above the average balance for their account_type. Show account_id, account_type, balance, and customer name.

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure for 'Banking'. The central pane shows a SQL query for Q27, which is highlighted with a yellow box. The query is as follows:

```

JOIN loans l ON c.customer_id = l.customer_id
WHERE l.status = 'Active'
GROUP BY c.customer_id
ORDER BY total_loans DESC LIMIT 1;

/*27. The wealth management team wants to identify accounts performing above their peer group.
Find accounts with a balance above the average balance for their account_type. Show account_id, account_type, balance, and customer name.*/
SELECT a.account_id, a.account_type, a.balance, c.first_name, c.last_name FROM customers c
JOIN accounts a ON c.customer_id = a.customer_id
WHERE a.balance > (select AVG(a2.balance) FROM accounts a2 WHERE a2.account_type = a.account_type)
ORDER BY a.account_id;

/*28. Rank accounts by balance within each account type using a window function.
Show account_id, account_type, balance, and balance_rank. Order by account_type, then rank.*/
SELECT account_id, account_type, balance, RANK() OVER (PARTITION BY account_type ORDER BY balance DESC) AS balance_rank
FROM accounts
ORDER BY account_type, balance_rank;

/*29. The branch management team wants to identify the top depositor at each location.
Show first_name, last_name, branch_name, and total_balance.*/

```

The bottom pane shows the 'Data Output' tab with the result of the query, highlighted with a blue box. The result is a table with 3 rows:

account_id	account_type	balance	first_name	last_name
5001	Checking	3500.5	Alice	Smith
5005	Checking	4200.25	David	Brown
5008	Savings	25000	Frank	Miller

The status bar at the bottom indicates 'Total rows: 3 Query complete 00:00:00.140'.

Q28. Rank accounts by balance within each account type using a window function. Show account_id, account_type, balance, and balance_rank. Order by account_type, then rank.

The screenshot shows the pgAdmin 4 interface. The SQL query editor contains the following query:

```

241     where sub.account_type = accounts.account_type
242 )
243 order by account_type, balance desc;
244
245 -- Q28. Rank accounts by balance within each account type using a window function. Show account_id, acco
246 select account_id, account_type, balance,
247        rank() over (partition by account_type order by balance desc) as balance_rank
248 from accounts
249 order by account_type, balance_rank;
250

```

The query results are displayed in the Data Output tab, showing 8 rows. The columns are account_id, account_type, balance, and balance_rank.

account_id	account_type	balance	balance_rank
1	5006	CD	15000
2	5005	Checking	4200.25
3	5001	Checking	3500.5
4	5003	Checking	2100.75
5	5007	Checking	1800
6	5008	Savings	25000
7	5002	Savings	12000
8	5004	Savings	8500

Successfully run. Total query runtime: 59 m

Q29. The branch management team wants to identify the top depositor at each location. Find the customer with the highest total account balance in each branch. Show first name, last name, branch name, and total_balance.

The screenshot shows the pgAdmin 4 interface. The SQL query editor contains the following query:

```

287 /*Q29: The branch management team wants to identify the top depositor at each location. Find the customer with the highest total account ba
288 */
289 WITH customer_balances AS (
290 SELECT c.customer_id, c.first_name, c.last_name, c.branch_id, SUM(a.balance) AS total_balance
291 FROM customers c
292 JOIN accounts a
293 ON c.customer_id = a.customer_id
294 GROUP BY c.customer_id, c.first_name, c.last_name, c.branch_id),
295 ranked AS (
296 SELECT *,
297        RANK() OVER (PARTITION BY branch_id
298                     ORDER BY total_balance DESC) AS rnk
299 FROM customer_balances)
300 SELECT r.first_name, r.last_name, b.branch_name, r.total_balance
301 FROM ranked r
302 JOIN branches b ON r.branch_id = b.branch_id
303 WHERE r.rnk = 1
304 ORDER BY b.branch_name;

```

The query results are displayed in the Data Output tab, showing 8 rows. The columns are account_id, account_type, flow_type, and transaction_date.

account_id	account_type	flow_type	transaction_date
1	5001	500	Income
2	5001	200	Expense
3	5002	1000	Income
4	5003	150	Expense
5	5004	250	Income

Total rows: 8 Query complete 00:00:00.159

Q30. The product team wants a breakdown of account distribution by balance tier. Using a CTE, classify accounts into tiers: High (balance ≥ 10000), Medium (balance ≥ 3000), Low (below 3000). Show each tier's account count, average balance, and total balance. Order by avg_balance descending.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'public' schema and various objects like Aggregates, Collations, Domains, FTS Configurations, FTS Dictionaries, FTS Parsers, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, Tables (5), Triggers, Types, and Views. The main pane shows a SQL query editor with the following code:

```

185 ORDER BY total_loans DESC
186 LIMIT 1;
187
188 -- Q30: The product team wants a breakdown of account distribution by balance tier.
189 --Using a CTE, classify accounts into tiers: High (balance >= 10000), Medium (balance >= 3000), Low (below 3000).
190 --Show each tier's account count, average balance, and total balance. Order by avg balance descending.
191
192 WITH account_tiers
193 AS (SELECT account_id, account_type, balance,
194 CASE
195 WHEN balance >= 10000 THEN 'High'
196 WHEN balance >= 3000 THEN 'Medium'
197 ELSE 'Low'
198 END AS tier
199 FROM accounts)
200 SELECT tier, COUNT(*) AS account_count, AVG(balance) AS avg_balance, SUM(balance) AS total_balance
201 FROM account_tiers
202 GROUP BY tier
203 ORDER BY avg_balance DESC;

```

The 'Data Output' pane shows the results of the query:

tier	account_count	avg_balance	total_balance
High	3	17333.3333333333	52000
Medium	3	5400.25	16200.75
Low	2	1950.375	3900.75

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.798'. The system tray shows the date '19/07/2026' and a temperature of '30°'.